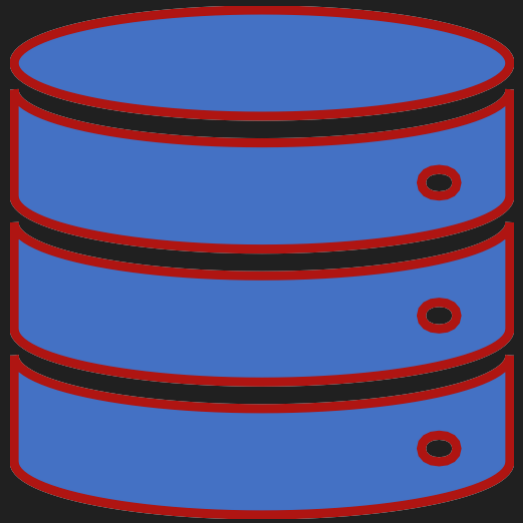


Correlated Subqueries

Correlated Subqueries

- A *correlated subquery* is a subquery that refers to a column of a table that is not in its FROM clause.
- The column can be in the Projection clause or in the WHERE clause.
- In general, correlated subqueries diminish performance.
- It is recommended that you qualify the column name in subqueries with the name or alias of the table, in order to remove any doubt regarding in which table the column resides.

Feature	Nested Subqueries	Correlated Subqueries
Execution	Inner query is executed once.	Inner query is executed multiple times, once for each row of the outer query.
Dependency	Inner query is independent of the outer query.	Inner query depends on the outer query for values.
Performance	Typically faster, especially for large datasets (because the subquery is executed once).	Can be slower because it runs the subquery repeatedly.
Usage	Commonly used for comparisons (e.g., IN, ANY, ALL, EXISTS).	Used for comparisons where rows from the outer query are needed in the inner query.
Example	Employee's salary compared to the company average.	Employee's salary compared to the department average.

Correlated Subqueries

- An example of a correlated subquery that returns a list of the 10 latest shipping dates in the orders table.
- It includes an ORDER BY clause after the subquery to order the results because (except in the FROM clause) you cannot include ORDER BY within a subquery.

```
SELECT po_num, ship_date FROM orders main WHERE 10 >
(SELECT COUNT (DISTINCT ship_date) FROM orders sub
WHERE sub.ship_date < main.ship_date) AND ship_date IS NOT
NULL ORDER BY ship_date, po_num;
```

Correlated Subqueries The EXISTS keyword

The keyword EXISTS is known as an *existential qualifier* because the subquery is true only if the outer SELECT, as the following query shows, finds at least one row.

```
SELECT UNIQUE manu_name, lead_time FROM manufact WHERE EXISTS (SELECT *
FROM stock WHERE description like '*shoe*' AND manufact.manu_code =
stock.manu_code);
```

Correlated Subqueries The EXISTS keyword

The keyword EXISTS is known as an *existential qualifier* because the subquery is true only if the outer SELECT, as the following query shows, finds at least one row.

```
SELECT UNIQUE manu_name, lead_time FROM manufact WHERE EXISTS (SELECT *
FROM stock WHERE description like '*shoe*' AND manufact.manu_code =
stock.manu_code);
```

Another method

```
SELECT UNIQUE manu_name, lead_time FROM stock, manufact WHERE
manufact.manu_code IN (SELECT manu_code FROM stock WHERE description like
'*shoe*') AND stock.manu_code = manufact.manu_code;
```

Subqueries in a Projection clause

A subquery can occur in the Projection clause of another SELECT statement.

The following query shows how you might use a subquery in a Projection clause to return the total shipping charges (from the **orders** table) for each customer in the **customer** table.

You could also write this query as a join between two tables.

```
SELECT customer.customer_num,
   (SELECT SUM(ship_charge)
       FROM orders
       WHERE customer.customer_num = orders.customer_num)
         AS total_ship_chg
   FROM customer;
```

Subqueries in the FROM clause

Return the results of a subquery that retrieves all fields of the **address** column in the **employee** table.

```
SELECT * FROM (SELECT address.* FROM employee);
```

Subqueries in the FROM clause

The outer query selects only the first qualifying row of a derived table that a subquery in the FROM clause specifies as a simple join on the **customer** and **cust_calls** tables.

```
SELECT LIMIT 1 * FROM (SELECT c.customer_num, c.lname, c.company,
c.phone, u.call_dtime, u.call_descr
FROM customer c, cust_calls u
WHERE
c.customer_num = u.customer_num
ORDER BY u.call_dtime DESC);
```

When to Use:

- When you need row-by-row comparisons.
- When the subquery needs to access data from the outer query.
- When checking for the existence of related data (EXISTS).
- When filtering based on complex conditions within the same or related tables.
- When finding duplicates or shared values across rows.

When to Use Nested and Correlated Subqueries:

•**Nested Subqueries**: Use them when you need a **single value** or **set of values** to compare against or filter data.

•**Correlated Subqueries**: Use them when you need to perform **row-by-row comparison**, where the subquery depends on each row of the outer query.

•**EXISTS with Correlated Subqueries**: Use EXISTS for checking the presence of related rows efficiently, especially when you don't need to retrieve the actual data from the subquery.

Summary:

•**Nested subqueries** are good for filtering or comparing with aggregate values.
•**Correlated subqueries** allow row-by-row comparisons when relationships need to be checked for each record.
•Subqueries offer simpler syntax in some cases, while JOINs are often more efficient for complex relationships or large datasets.

Thank You